

TWO DIMENSIONAL PATTERN MATCHING

R.S. BIRD

*Department of Computer Science, University of Reading, Whiteknights Park,
Reading, Berkshire, U.K.*

Received 5 July 1977

Pattern recognition, syntactic analysis, text editing

1. Introduction

The Knuth-Morris-Pratt algorithm [3] solves the following string matching problem: given a string of symbols $p_1 p_2 \dots p_n$, called the *pattern*, and a second string $t_1 t_2 \dots t_m$, called the *text*, to determine for each j whether or not $t_{j-n+1} \dots t_j = p_1 p_2 \dots p_n$. The KMP algorithm possesses the following noteworthy properties:

- (1) its running time is $O(n + m)$, where the coefficient of proportionality is small and independent of the size of the alphabet;
- (2) its space requirements are $O(n)$, so no part of the text, which may be substantially longer than the pattern, need be kept in store;
- (3) its on-line nature, whereby the outcome for value j is determined before t_{j+1} is read.

A natural generalisation of this problem is one in which the pattern is a two-dimensional *array* of symbols p_{ij} , $1 \leq i, j \leq n$, and the text is a second array t_{ij} , $1 \leq i, j \leq m$. In this case the problem is to determine where, if anywhere, the pattern occurs as a subarray of the text. Our purpose is to give an algorithm for the two dimensional case, one which follows the general approach of the KMP, and indeed uses the KMP as a subprogram. In particular, the algorithm has a running time of $O(n^2 + m^2)$, which is clearly optimal since both the pattern and the text have to be read and this takes $O(n^2 + m^2)$. The algorithm needs $O(n^2 + m)$ space, and also works on-line (under the assumption that the text is read row by row).

For reasons of space we assume some familiarity with

the KMP as described in [3], and also with the contents of [1] where a relevant extension is discussed. A related array matching problem is described and solved in [2].

2. The algorithm

The general scheme of the algorithm is similar to that of the KMP and is as follows:

process the pattern to yield relevant tables of information;

for $j = 1$ to m do

for $k = 1$ to m do

stage (j, k) , which is to use the tables to determine whether or not the pattern occurs as a subarray of the text with its bottom right hand corner (i.e. p_{nn}) on t_{jk} .

The computation in stage (j, k) is composed of two distinct parts, a *row-matching* step and a *column matching* step. We describe each step separately, and then show how they are combined in the final algorithm.

row-matching

The purpose of the row-matching step is to determine which row of the pattern, if any, matches a terminal substring of $t_{j_1} t_{j_2} \dots t_{j_k}$. This can be done using Aho and Corasick's algorithm for simultaneously matching a number of (linear) patterns with a text string. The idea (which is in fact a generalisation of the

KMP) is to consider each row of the pattern as a separate keyword and construct a pattern matching machine as described in [1]. We shall just give an example of this construction as full details are given in [1]. Consider the 5 X 5 pattern whose rows are $p_1 = aabba$, $p_2 = aaabb$, $p_3 = ababa$, $p_4 = aabba$, $p_5 = aaabb$ (note: $p_4 = p_1$ and $p_5 = p_2$). The pattern matching machine in this case consists of a goto function, as shown in Fig. 1 and a failure function as shown in Table 1.

The goto function g maps a pair consisting of a state and an input symbol into a state or the message *fail*, and is pictured as a directed graph. Thus, for example, $g(3, b) = 4$ and $g(3, a) = fail$. The failure function f , which maps states into states, is consulted whenever the goto function reports *fail*. States 5, 8 and 12 are special in that whenever they are reached an appropriate row of the pattern has been found, in fact row p_1 for state 5, row p_2 for state 8, and row p_3 for state 12. (Rows p_4 and p_5 will never be selected as they are the same as rows p_1 and p_2 respectively). We can represent this information by two further tables, a boolean *terminal (state)* which indicates which states are terminal, and a function *row (state)* which gives the row corresponding to a terminal state. We use the pattern matching machine on the text in the following way:

```
for j = 1 to m do
  begin state ← 0;
    for k = 1 to m do
      begin read x; comment x = tjk;
        while g(state, x) = fail do state ← f(state);
        state ← g(state, x);
        if terminal (state) then print row (state) fi
      end
    end
end
```

As explained in [1] the construction of the pattern matching machine takes $O(n^2)$ steps and the above algorithm runs in $O(m^2)$ steps.

column-matching

Discovering that row p_i of the pattern occurs in a given place in the text is clearly not enough, we need also to be able to determine whether or not rows

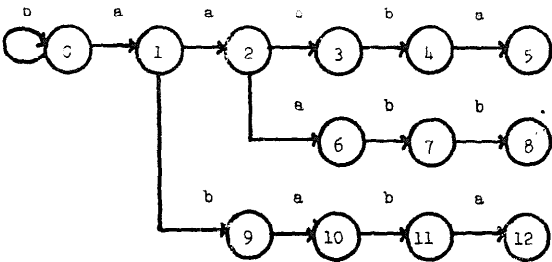


Fig. 1.

$p_1, p_2 \dots p_{i-1}$ occur immediately above row p_i in order to say whether or not the complete pattern is a subarray of the text. We can do this quite neatly by using the KMP algorithm again, this time vertically as it were. Say we maintain a vector $a[1], \dots a[m]$ of values, such that at the beginning of each step (j, k) the fact that $a[k] = i$ means just that rows $p_1 \dots p_{i-1}$ of the pattern have been discovered to match the text in positions $t(j - i + 1, k - n + 1) \dots t(j - i + 1, k), \dots, t(j - 1, k - n + 1) \dots t(j - 1, k)$. This situation can be pictured as shown in Fig. 2. Now, by determining the proper value of $a[k]$ at the end of step (j, k) we can solve our problem, for we have $a[k] = n + 1$ just in the case that all rows of the pattern have been matched. Certainly, if the row-matching stage for step (j, k) fails to yield a match for any row of the pattern, then we can reset $a[k] := 1$. On the other hand, if the row-matching discovers that row p_r matches the text up to t_{jk} , then we have to set $a[k] = s + 1$, where s is the maximum value $\leq i$ such that

$$p_r = p_s, p_{i-1} = p_{s-1}, \dots, p_{i-s+1} = p_1.$$

We can compute s by using the KMP applied to the (linear!) pattern $p_1, p_2, \dots, p_n$, regarding each p_i as a *single* symbol. These are not necessarily distinct symbols, as some rows may be the same as others. In fact, for our example pattern above, we have $p_4 = p_1$ and $p_5 = p_2$. We can map rows into the *symbols* 1, 2 ..., so our example pattern gives the string 12312, and this is the string we have to match for. The KMP uses a failure function t , which for the example gives Table 2. Thus

Table 1

i	1	2	3	4	5	6	7	8	9	10	11	12
$f(i)$	0	1	0	0	1	2	0	0	0	1	0	1

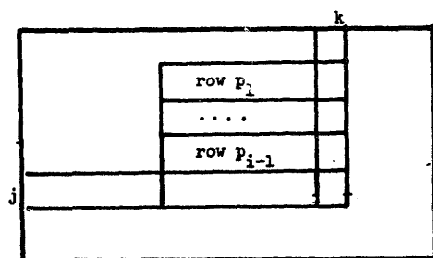


Fig. 2.

we can update the array a , and so solve our problem, by replacing the line **if terminal (state) then print row (state) fi** in the row-matching stage, by the code

```

if terminal (state)
  then  $i := a[k]$ ;
    while  $i > 0$  and  $p(i) \neq \text{row (state)}$  do  $i := t(i)$ 
     $a[k] := i + 1$ ;
    if  $i = n$  then print pattern found at  $(j, k)$ 
  else  $a[k] := 1$ 
fi

```

For this to work correctly, we have to initialise $a[k] = 1$ for $1 \leq k \leq m$ at the beginning of the algorithm. Notice that this procedure of column matching only works because the KMP algorithm is independent of the size of the symbol alphabet, which can consequently be arbitrarily large. The calculation of the row string $p(1), \dots, p(n)$ and its failure function t can clearly be done in $O(n^2) + O(n) = O(n^2)$ steps, and the running time of the above section of the program is $O(m^2 + mn)$, since

Table 2

row	1	2	3	4	5
p	1	2	3	1	2
t	0	1	1	1	2

there are m columns and the overall time spent on each column is $O(m + n)$. Thus, since $O(mn) = O(m^2 + n^2)$, the complete algorithm has a running time $O(m^2 + n^2)$. Furthermore, to store the relevant tables of functions and the array a , the algorithm needs $O(n^2 + m)$ space.

Acknowledgement

The author would like to thank Dr. Leonard Dresel for suggesting the problem.

References

- [1] Aho, A.V., and Corasick, M.J., Efficient string matching, C. ACM (1975) 18 (6) 333-340.
- [2] Karp, R.M., Miller, R.E., and Rosenberg, A.L., Rapid identification of repeated patterns in strings, trees and arrays. Proc. 4th Annual ACM Symposium on Theory of Computing (1972) 125-136.
- [3] Knuth, D.E., Morris, J.H. Jr., and Pratt V.R., Fast pattern matching in strings, Research report STAN-CS-74-440, Computer Science Department, Stanford, California (1974).